

01. Windows 入⼝ PowerShell ルータ

solar_ws0129-main

2026-07-29

Contents

1 対象	2
2 このファイルを読む理由	2
3 呼び出し関係	2
3.1 どこから呼ばれるか	2
3.2 次に読むと理解が進むファイル	2
3.3 同一リポジトリ内の主要依存	2
4 ソース冒頭の把握	2
4.1 代表コード断片	2
5 主要構造の読み取り	4
5.1 ファイルを上から読んだときの定義順	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 CLI、PowerShell、Bash、環境変数、終了コード	5
6.3 例外、try/finally、with、資源解放	6
6.4 天候、route、補間、時刻、単位	6
6.5 freshness、filter、guard、fallback、fail-safe	6
6.6 launch Action、Node Action、実行可能名、remapping	6
6.7 単体試験、SILS、replay、観測可能性	7
7 関数・クラスを上から順に解説	7
7.1 shell 分岐と外部コマンド	7
8 処理の流れ	8
9 このファイルの位置づけ	8

1 対象

項目	内容
ファイル	SolarSim.ps1
ソース SHA-256	fa039bfb741ab4719218ffc48738f42ce3ea6f7c3ccf33dd0c4987ea87f29c1
種別	PowerShell
区分	入口
役割	Windows PowerShell から build、launch、offline script、Grafana 操作をまとめて呼び出す最上位入口。
起動文脈	オペレータが最初に叩くコマンド入口。

2 このファイルを読む理由

Windows PowerShell から build、launch、offline script、Grafana 操作をまとめて呼び出す最上位入口。

- WSL ディストリビューションを解決する。
- Action と Mode を shell 側へ受け渡す。
- dashboard や report の自動オープンも担う。

3 呼び出し関係

3.1 どこから呼ばれるか

- 人間の運用操作

3.2 次に読むと理解が進むファイル

- scripts/solar_control.sh
- launch/solar_race_live_wifi.launch.py
- scripts/solar_sim.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
param(
    [ValidateSet('up', 'build', 'start', 'stop', 'restart', 'status', 'graph', 'simulate', '
historical-weather', 'historical-simulate', 'forecast', 'identify', 'fit', 'learn', '
audit', 'package', 'blank', 'log', 'grafana', 'grafana-stop')]
```

```

    [string]$Action = 'up',
    [ValidateSet('sim', 'measure', 'live', 'live_wifi')]
    [string]$Mode = 'sim',
    [string]$Profile = 'project_packages/bwsc2025_fitted_mle19_energywindow_inertia/profile.
yaml',
    [string]$Distro = '',
    [switch]$Help
)

$errorActionPreference = 'Stop'

if ($Help) {
    @'
SolarSim.ps1 -Action <action> [-Mode <mode>] [-Profile <profile.yaml>] [-Distro <WSL name>]

Actions: up, build, start, stop, restart, status, graph, simulate, forecast,
        identify, fit, learn, audit, package, blank, log, grafana, grafana-stop
Modes:   sim, measure, live, live_wifi

Examples:
.\SolarSim.ps1 -Action build
.\SolarSim.ps1 -Action up -Mode sim
.\SolarSim.ps1 -Action simulate -Profile project_packages/<vehicle>/profile.yaml
'@ | Write-Host
    exit 0
}

function Normalize-WslText {
    param([string]$Text)
    if ($null -eq $Text) {
        return ''
    }
    return ([string]$Text).Replace("`0", '').Trim()
}

function Resolve-WslDistro {
    param([string]$Preferred)
    if ($Preferred) {
        return (Normalize-WslText $Preferred)
    }
    $installed = (& wsl.exe -l -q) | ForEach-Object { Normalize-WslText $_ } | Where-Object
    { $_ }
    foreach ($candidate in @('Ubuntu-22.04', 'Ubuntu', 'Ubuntu-24.04')) {
        if ($installed -contains $candidate) {
            return $candidate
        }
    }
    if ($installed.Count -gt 0) {
        return $installed[0]
    }
    throw 'WSL distro was not found.'
}

function Resolve-WslArgPath {
    param(
        [string]$TargetDistro,
        [string]$PathValue
    )
    if ([string]::IsNullOrEmpty($PathValue)) {
        return ''
    }
    if ([System.IO.Path]::IsPathRooted($PathValue)) {

```

```

        return (& wsl.exe -d $TargetDistro wslpath -a $PathValue).Trim()
    }
    return ($PathValue -replace '\\', '/')
}

function Invoke-WslRepoCommand {
    param(
        [string]$RepoRoot,

```

5 主要構造の読み取り

action 分岐は 16 件。

5.1 ファイルを上から読んだときの定義順

行	内容
1	param(
2	[ValidateSet('up', 'build', 'start', 'stop', 'restart', 'status', 'graph', 'simulate', 'historical-weather', 'historical-simulate', 'forecast', 'identify', 'fit', 'learn', 'audit', 'package', 'blank', 'log', 'grafana', 'grafana-stop')]
3	[string]\$Action = 'up',
4	[ValidateSet('sim', 'measure', 'live', 'live_wifi')]
5	[string]\$Mode = 'sim',
6	[string]\$Profile = 'project_packages/bwsc2025_fitted_mle19_energywindow_inertia/profile.yaml',
7	[string]\$Distro = "",
8	[switch]\$Help
9	)
11	\$ErrorActionPreference = 'Stop'
13	if (\$Help) {
14	@'
15	SolarSim.ps1 -Action <action> [-Mode <mode>] [-Profile <profile.yaml>] [-Distro <WSL name>]
17	Actions: up, build, start, stop, restart, status, graph, simulate, forecast,
18	identify, fit, learn, audit, package, blank, log, grafana, grafana-stop
19	Modes: sim, measure, live, live_wifi
21	Examples:
22	.\SolarSim.ps1 -Action build
23	.\SolarSim.ps1 -Action up -Mode sim
24	.\SolarSim.ps1 -Action simulate -Profile project_packages/<vehicle>/profile.yaml
25	'@ Write-Host
26	exit 0
27	}
29	function Normalize-WslText {
30	param([string]\$Text)
31	if (\$null -eq \$Text) {
32	return ""
33	}
34	return ([string]\$Text).Replace("'", "").Trim()
35	}

```

37     function Resolve-WslDistro {
38     param([string]$Preferred)
39     if ($Preferred) {
40     return (Normalize-WslText $Preferred)
41     }
42     $installed = (& wsl.exe -l -q) | ForEach-Object { Normalize-WslText $_ } | Where-Object { $_ }
43     foreach ($candidate in @(‘Ubuntu-22.04’, ‘Ubuntu’, ‘Ubuntu-24.04’)) {
44     if ($installed -contains $candidate) {
45     return $candidate
46     }

```

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```

node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。

```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.3 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.4 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.5 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.6 launch Action、Node Action、実行可能名、remapping

‘launch_ros.actions.Node(...)’は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

‘DeclareLaunchArgument’は launch 実行時に受け取る入力欄を宣言し、‘LaunchConfiguration’はその値を後で解決する substitution を表す。‘perform(context)’は実行時 context から確定文字列を取り出す。

launch の ‘name’は Node 名 override、‘parameters’は起動時 parameter、‘remappings’は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

6.7 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 公式: Executors](#)

7 関数・クラスを上から順に解説

このファイルでは独立した関数・クラス定義が少ない。

7.1 shell 分岐と外部コマンド

行	種別	内容
250	action	up
278	action	build
281	action	start
284	action	stop
287	action	restart
290	action	status
293	action	graph
299	action	simulate
306	action	historical-weather
309	action	historical-simulate
317	action	forecast
320	action	identify
323	action	fit
326	action	learn
329	action	audit
332	action	log
42	command	<code>\$installed=(&wsl.exe-l-q) ForEach-Object{Normalize-WslText\$_} } Where-Object{\$_}</code>
63	command	<code>return(&wsl.exe-d\$TargetDistrowslpath-a\$PathValue) .Trim()</code>
76	command	<code>\$wslRepo=(&wsl.exe-d\$TargetDistrowslpath-a\$RepoRoot) .Trim()</code>

81	command	&wsl.exe -d \$TargetDistrobash -lc "cd '\$wslRepo' && bash ' ./scripts/solar_control.sh' '\$Command' '\$TargetMode' '\$wslProfile'"
207	command	if (-not (Get-Command docker -ErrorAction SilentlyContinue)) {
214	command	&docker compose up -d
216	command	throw "docker compose up failed with exit code \$LASTEXITCODE"
220	command	&docker compose down
222	command	throw "docker compose down failed with exit code \$LASTEXITCODE"
235	command	&python \$builder --force
257	command	if (Get-Command docker -ErrorAction SilentlyContinue) {
260	command	&docker compose up -d
262	command	throw "docker compose up failed with exit code \$LASTEXITCODE"

8 処理の流れ

1. PowerShell 引数を解釈する。
2. WSL 側のパスへ変換する。
3. scripts/solar_control.sh を action と mode 付きで呼ぶ。
4. 必要に応じて dashboard、graph、report を開く。

9 このファイルの位置づけ

この資料群では、SolarSim.ps1 を **入口** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の profile / launch / telemetry と後段の planner / logger / report へ繋がる。